Endpoint PowerShell Deployment & Patch Automation Toolkit

Implementation Solution Guide

Version 1.0.0 · 2026-06-28

Contents

Executive Summary — Endpoint PowerShell Deployment & Patch Automation Toolkit 4
- What you get . . . 4
- What this delivers . . . 4
- The repeatable deployment model . . . 4
- Outcome . . . 4

1. Solution Overview — Endpoint PowerShell Deployment & Patch Automation Toolkit 4
- What this toolkit is . . . 5
- Why this approach (and why it pays off) . . . 5
- How it works — the detect-then-remediate pattern . . . 5
- What you can automate with it . . . 5
- Safe-by-default design . . . 6
- How results are reported . . . 6
- Named objects this toolkit creates . . . 6
- Limitations (be honest with buyers) . . . 6
- The repeatable deployment model . . . 7

2. Requirements Guide — Endpoint PowerShell Deployment & Patch Automation Toolkit 7
- 2.1 Intune Remediations entitlement . . . 7
- 2.2 Microsoft Graph PowerShell SDK . . . 7
- 2.3 Microsoft Graph permissions . . . 7
- 2.4 PowerShell execution policy and 64-bit run . . . 7
- 2.5 Script signing (if required by policy) . . . 8
- 2.6 A results destination (optional) . . . 8
- Checklist . . . 8

3. Implementation Guide — Endpoint PowerShell Deployment & Patch Automation Toolkit 8
- Step 1 — Create the device groups . . . 9
- Step 2 — Stand up and review the toolkit locally . . . 9
- Step 3 — Create the detect/remediate remediation . . . 9
- Step 4 — Assign and schedule the remediation . . . 10
- Step 5 — Add a patch-state check remediation (optional) . . . 10
- Step 6 — Wire deployment / cleanup automation . . . 10
- Step 7 — Validate on the pilot . . . 10
- Step 8 — Expand to production . . . 10

5. Testing Guide — Endpoint PowerShell Deployment & Patch Automation Toolkit 11
- Test 1 — Toolkit readiness . . . 11
- Test 2 — Detection logic is correct . . . 11

Test 3 — Remediation is safe by default . . . 11
Test 4 — The remediation runs in Intune . . . 11
Test 5 — Schedule fires . . . 11
Test 6 — Deployment / cleanup runner . . . 12
Test 7 — Fleet report . . . 12
Pass/fail gate . . . 12
6. Troubleshooting Guide — Endpoint PowerShell Deployment & Patch Automation Toolkit 12
A. Remediation never runs / no results . . . 12
B. Detection always reports non-compliant . . . 12
C. Remediation does not change anything . . . 13
D. Script blocked from running . . . 13
E. Fleet report errors . . . 13
F. Deployment / cleanup runner issues . . . 13
G. Diagnostics quick reference . . . 13
H. When to escalate . . . 14
7. Best Practices — Endpoint PowerShell Deployment & Patch Automation Toolkit 14
8. Deliverables Summary — Endpoint PowerShell Deployment & Patch Automation Toolkit 15
Documentation . . . 15
PowerShell scripts . . . 15
What the buyer receives . . . 15
Outcome . . . 15
Appendix A — PowerShell Scripts 16

Executive Summary — Endpoint PowerShell Deployment & Patch Automation Toolkit

Endpoint engineers lose hours every week to the same manual chores — remediating drifted settings, confirming patch state, deploying or removing software, and cleaning up after failed installs — done by hand, a little differently every time. The result is inconsistent devices, no record of what was done, and work that doesn't scale past a handful of machines.

This is a complete, deploy-it-today toolkit for automating endpoint deployment and patching with PowerShell and Intune remediations (proactive remediations), written for an administrator with no prior automation experience. It turns repeated manual tasks into tested, parameterized scripts that run on a schedule, report their results, and make every device converge to the same known-good state.

What you get

- A plain-language Solution Overview of the automation model: the detect-then-remediate pattern,

scheduled runs, safe-by-default design, and how results are reported.

- A Requirements Guide with verify / implement-if-missing for every prerequisite.
- An Implementation Guide of precise, click-by-click steps in the required order.
- Five PowerShell scripts for automation-readiness discovery, a detection/validation script, a

fleet automation-results report, a safe remediation script (-WhatIf default), and a deployment/cleanup runner (-WhatIf default).

- Testing and Troubleshooting guides for a confident, gated rollout.

What this delivers

A repeatable automation model — detect-then-remediate jobs deployed as Intune remediations that run on a schedule, scheduled patch-state checks, deployment and cleanup runners that are safe by default, and fleet reporting that shows which devices have converged and which are lagging. Every script is parameterized and tested, so the same automation runs everywhere the same way.

The repeatable deployment model

Assess → Configure → Pilot → Validate → Expand — with a pilot gate before any fleet rollout.

Outcome

The repetitive endpoint work automated — detect-then-remediate jobs running on a schedule, results reported, every device converging to one known-good state — deployable by a single administrator and proven on a pilot before you scale.

1. Solution Overview — Endpoint PowerShell Deployment & Patch Automation Toolkit

What this toolkit is

This is a tested set of PowerShell scripts and Intune remediations (also called proactive remediations) that automate the repetitive endpoint work an engineer otherwise does by hand: remediating drifted settings, confirming patch state, deploying or removing software, and cleaning up after failed installs. Instead of one-off manual fixes, you deploy parameterized scripts that run on a schedule, report what they did, and bring every device to the same known-good state.

Three building blocks work together:

- Intune remediations (proactive remediations) — a paired detection script and remediation script that Intune runs on enrolled devices on a schedule. The detection script decides whether a device is healthy; only if it is not does Intune run the remediation script.
- Microsoft Graph PowerShell — used from an admin workstation to read fleet state (device inventory, remediation results) so you can report across every device.
- Scheduled local scripts — the same tested scripts, runnable on demand or on a schedule, for tasks that fit a runner better than a remediation pair (for example a deployment or a cleanup sweep).

Why this approach (and why it pays off)

Manual endpoint work does not scale and is never consistent. Two admins fix the same problem two different ways; nobody can prove what was changed; and the next time the problem recurs, the work starts over. The detect-then-remediate pattern fixes this: you write the desired state once, Intune checks it everywhere on a schedule, fixes only the devices that have drifted, and records the result. The work becomes assign-once, repeat-everywhere — every device converges to one definition of healthy, and you get a fleet-wide record instead of a memory of what you typed last Tuesday.

How it works — the detect-then-remediate pattern

1. A detection script inspects the device and exits `0` (compliant — nothing to do) or `1` (non-compliant — remediation needed). It writes a short status line that Intune captures.
1. When detection exits `1`, Intune runs the paired remediation script, which makes the change and exits `0` on success.
1. Intune runs the pair on a schedule (hourly, daily, or once) under the SYSTEM account (or the logged-on user, if you choose), so devices are re-checked continuously and self-heal.
1. Results — detection state, remediation state, output, and timing — roll up in Intune under the remediation, and can be read fleet-wide with Microsoft Graph.

What you can automate with it

- Configuration drift — detect a wrong/missing setting (registry value, service state, scheduled task) and remediate it back to the standard.

• Patch-state checks — detect devices missing a required update or below a target build and flag or trigger a scan.

• Software deployment and removal — run a parameterized deployment/cleanup runner to install, repair, or uninstall an app and tidy up leftover files after a failed install.

• Cleanup after failures — remove orphaned folders, stale temp data, or partial installs that block the next attempt.

Safe-by-default design

The remediation script and the deployment/cleanup runner default to `-WhatIf`: on first run they report exactly what they would change without changing anything. You review the planned actions on a pilot, then re-run with `-WhatIf:$false` (or the toolkit's `-Apply` switch) to make the change. Every script writes a clear status line and a non-zero exit code on failure so Intune and your reports catch problems early.

How results are reported

• Per device, in Intune, under each remediation: detection/remediation status, output, and run time.

• Fleet-wide, with `PSAUTO-Report.ps1` (Microsoft Graph), which summarizes remediation results across every targeted device so you can see how many have converged and chase the laggards.

• Locally, every script prints a readable status line and returns a success/failure exit code.

Named objects this toolkit creates

Name every object you create with the convention `<Purpose> – <Scope> – <Stage>` and reuse the names verbatim across Intune, your groups, and your documentation:

• Remediation package (detect/remediate pair): `Endpoint Drift Remediation – Windows – Production`

• Patch-state check remediation: `Patch State Check – Windows – Production`

• Pilot device group: `Endpoint Automation Pilot – Windows – Pilot`

• Production device group: `Endpoint Automation Broad – Windows – Production`

• Remediation schedule: `Endpoint Automation Schedule – Windows – Daily`

Limitations (be honest with buyers)

• Intune remediations require Intune Plan 1 plus the Remediations entitlement (included in Windows Enterprise E3/E5, Education A3/A5, or the Intune add-on) — see the Requirements Guide.

• Remediation scripts run on Windows; they are not a cross-platform configuration manager.

• Each remediation script must finish quickly (Intune enforces a timeout) and run non-interactively under SYSTEM by default — design scripts accordingly.

- This toolkit automates tasks you define; it is not a substitute for an application-patching product for third-party software at scale.

The repeatable deployment model

> *Stand up the toolkit → deploy a detect/remediate pair as an Intune remediation → schedule it → assign to the pilot → validate → expand to production. The repeatable way: Assess → Configure → Pilot → Validate → Expand.*

2. Requirements Guide — Endpoint PowerShell Deployment & Patch Automation Toolkit

Verify **every** requirement before configuring. Each entry: Description · Why · Verify · Implement-if-missing. Category-wide prerequisites are in `global-requirements-infrastructure.md`.

2.1 Intune Remediations entitlement

Description: A license that includes Intune Remediations (proactive remediations). Why: The detect-then-remediate scheduled scripts are an Intune Remediations feature; without the entitlement the Remediations blade and scheduled runs are unavailable. Verify: Sign in to `https://intune.microsoft.com`, go to Devices → Manage devices → Remediations (Scripts and remediations); confirm you can open it and click `Create script package`. Implement: Assign a qualifying license — Windows 10/11 Enterprise E3 or E5, Education A3 or A5, or the standalone Intune Plan 1 with the Remediations add-on — to the targeted users/devices.

2.2 Microsoft Graph PowerShell SDK

Description: The Microsoft Graph PowerShell SDK on the admin workstation. Why: The fleet report and the rings/assignment audit read Intune data through Microsoft Graph. Verify: Run `Get-Module -ListAvailable Microsoft.Graph.Authentication`; run `PSAUTO-Discovery.ps1`, which checks for the module. Implement: Run `Install-Module Microsoft.Graph -Scope CurrentUser`.

2.3 Microsoft Graph permissions

Description: Delegated Graph scopes to read device and remediation data. Why: The report scripts call Graph for managed-device inventory and remediation results. Verify: When you run `PSAUTO-Report.ps1` the consent prompt lists `DeviceManagementManagedDevices.Read.All` and `DeviceManagementConfiguration.Read.All`. Implement: Consent at first sign-in as an Intune Administrator, or have an admin grant the delegated scopes to the Microsoft Graph PowerShell app.

2.4 PowerShell execution policy and 64-bit run

Description: Scripts can run, and remediations run in the 64-bit PowerShell host. Why: Intune can run remediation scripts in 32-bit by default; registry and file paths differ between 32-bit and 64-bit, which causes false "non-compliant" results. Verify: Run `Get-ExecutionPolicy -List`; in the remediation's

settings confirm `Run script in 64-bit PowerShell` is set to `Yes`. Implement: Set the remediation's `Run script in 64-bit PowerShell` to `Yes`; for local runs use `-ExecutionPolicy Bypass`. Sign your scripts if your policy requires it (see 2.5).

2.5 Script signing (if required by policy)

Description: A code-signing certificate, if your execution policy is `AllSigned`. Why: An `AllSigned` policy refuses unsigned scripts; remediations would fail to run. Verify: Run `Get-ExecutionPolicy`; if it returns `AllSigned`, scripts must be signed. Implement: Sign each `.ps1` with `Set-AuthenticodeSignature` using a trusted code-signing certificate, or set the remediation to `Enforce script signature check = No` only where policy allows.

2.6 A results destination (optional)

Description: A folder, share, or Log Analytics workspace for run logs, if you want a durable record beyond Intune's built-in results. Why: Intune retains remediation results, but a central log makes auditing and trending easier. Verify: The chosen folder/share is writable by SYSTEM, or the Log Analytics workspace exists. Implement: Create the share/workspace and point the deployment/cleanup runner's `-LogPath` at it.

Checklist

- Intune Remediations entitlement (Remediations blade opens)
- Microsoft Graph PowerShell SDK installed — `PSAUTO-Discovery.ps1`
- Graph scopes consented (managed devices + configuration read)
- Execution policy allows the scripts; remediations set to run 64-bit
- Scripts signed if policy is `AllSigned`
- (Optional) Results destination chosen for run logs
- Pilot + Production device groups created

3. Implementation Guide — Endpoint PowerShell Deployment & Patch Automation Toolkit

Follow these steps **in order**, in the Microsoft Intune admin center (`https://intune.microsoft.com`).

> *Model: deploy a detect/remediate pair as an Intune remediation, run it on a schedule against a small pilot, validate, then expand to the fleet. The pilot is the gate before any production rollout.*

Reuse these object names verbatim (convention `<Purpose> – <Scope> – <Stage>`):

- Remediation package: `Endpoint Drift Remediation – Windows – Production`
- Patch-state remediation: `Patch State Check – Windows – Production`
- Pilot group: `Endpoint Automation Pilot – Windows – Pilot`

- Production group: `Endpoint Automation Broad – Windows – Production`
- Schedule: `Endpoint Automation Schedule – Windows – Daily`

Step 1 — Create the device groups

1. Go to Entra → Groups → New group → Security → name `Endpoint Automation Pilot – Windows – Pilot`

(a few representative devices). Click `Create`.

1. Create a second group named `Endpoint Automation Broad – Windows – Production` (the rest of the

fleet). Click `Create`.

Step 2 — Stand up and review the toolkit locally

1. On your admin workstation, unzip the five scripts to a working folder.
2. Run `PSAUTO-Discovery.ps1` to confirm automation readiness (Graph SDK present, execution policy,

64-bit host, remediations reachable).

1. Open `PSAUTO-Validation.ps1` (the sample detection script) and `PSAUTO-Remediation.ps1` (the

paired remediation) and set the desired-state values for your environment at the top of each file.

1. Run `PSAUTO-Remediation.ps1` once — it defaults to `-WhatIf` and prints exactly what it would

change, changing nothing. Confirm the planned actions look correct.

Step 3 — Create the detect/remediate remediation

1. In Intune go to Devices → Manage devices → Remediations (Scripts and remediations).
2. Click `Create script package`.
3. Basics: Name `Endpoint Drift Remediation – Windows – Production`; add a short description. Click

`Next`.

1. Settings:

- `Detection script file`: upload `PSAUTO-Validation.ps1`.
- `Remediation script file`: upload `PSAUTO-Remediation.ps1` (set its `-Apply` default, or leave

`-WhatIf` for a report-only first pass — see Step 6).

- `Run this script using the logged-on credentials`: `No` (run as SYSTEM).
- `Enforce script signature check`: `No` (or `Yes` if you signed the scripts — see Requirements 2.5).

- `Run script in 64-bit PowerShell`: `Yes`.

Click `Next`.

Step 4 — Assign and schedule the remediation

1. On the `Assignments` tab, click `Add groups` and add `Endpoint Automation Pilot – Windows – Pilot`.
2. Next to the assigned group set the schedule for `Endpoint Automation Schedule – Windows – Daily`:

choose `Daily` and a time (or `Hourly` / `Once` to suit the task).

1. Click `Next`, review, and click `Create`.

Step 5 — Add a patch-state check remediation (optional)

1. Repeat Step 3 with a detection script that flags devices missing a required update or below a

target build, and a remediation that triggers a Windows Update scan.

1. Name it `Patch State Check – Windows – Production`, assign to the pilot group, and set the same

daily schedule. Click `Create`.

Step 6 — Wire deployment / cleanup automation

1. For software deploy/remove and post-failure cleanup, use `PSAUTO-Deploy-CleanupRunner.ps1`.
2. Run it first with its default `-WhatIf` on a pilot device to confirm the planned install/uninstall

and cleanup actions, then re-run with `-WhatIf:$false` to apply.

1. Optionally point `-LogPath` at your results destination (Requirements 2.6) for a durable record,

and schedule it as its own remediation pair or local scheduled task.

Step 7 — Validate on the pilot

1. On a pilot device, force an Intune sync (Settings → Accounts → Access work or school → Info →

Sync), or wait for the schedule.

1. In Intune open `Endpoint Drift Remediation – Windows – Production` → Device status; confirm pilot

devices show a detection result and, where they had drifted, a successful remediation.

1. Run `PSAUTO-Report.ps1` to confirm the result fleet-wide. Complete the Testing Guide.

Step 8 — Expand to production

1. Once the pilot passes the Testing Guide gate, edit the remediation's `Assignments` and add

`Endpoint Automation Broad – Windows – Production`.

1. Keep the same `Endpoint Automation Schedule – Windows – Daily` schedule.
2. Re-validate each cycle with `PSAUTO-Report.ps1` and chase any devices that have not converged.

Order of operations

`Groups → review toolkit → create detect/remediate → assign + schedule → (patch check) → deploy/cleanup runner → validate (pilot gate) → expand`

5. Testing Guide — Endpoint PowerShell Deployment & Patch Automation Toolkit

Run after the remediation applies to the pilot (Implementation Step 7) and after any change to a detection or remediation script.

Test 1 — Toolkit readiness

Do: On the admin workstation run `PSAUTO-Discovery.ps1`. Expected: Graph SDK present, execution policy allows the scripts, 64-bit host confirmed, and the Remediations feature reachable — no blocking warnings.

Test 2 — Detection logic is correct

Do: On a known-good pilot device run `PSAUTO-Validation.ps1` directly; then on a deliberately drifted device run it again. Expected: Exit `0` (compliant) on the good device and exit `1` (non-compliant) on the drifted one, each with a clear status line.

Test 3 — Remediation is safe by default

Do: Run `PSAUTO-Remediation.ps1` with no parameters on a drifted pilot device. Expected: It reports the change it would make and changes nothing (`-WhatIf` default); re-running with `-WhatIf:$false` (or `-Apply`) makes the change and exits `0`.

Test 4 — The remediation runs in Intune

Do: In Intune open `Endpoint Drift Remediation – Windows – Production` → Device status after the pilot syncs. Expected: Pilot devices show a detection result; drifted devices show `Remediated`, healthy devices show `Without issues`.

Test 5 — Schedule fires

Do: Confirm the assignment uses `Endpoint Automation Schedule – Windows – Daily`; wait for or force the next run. Expected: The run date/time updates on the next cycle; devices are re-checked on schedule.

Test 6 — Deployment / cleanup runner

Do: Run `PSAUTO-Deploy-CleanupRunner.ps1` on a pilot device with its default `-WhatIf`, then with `-WhatIf:$false`. Expected: The planned install/uninstall and cleanup are reported, then applied; a log is written to `-LogPath` if set; the runner exits `0` on success.

Test 7 — Fleet report

Do: Run `PSAUTO-Report.ps1`. Expected: A per-device summary of remediation status (remediated / without issues / failed) across the targeted devices, plus a count, and an optional CSV.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–4 pass; the remediation remediates a drifted pilot device cleanly
Production pass	Tests 4, 5, 7 pass for the production group before/while expanding

6. Troubleshooting Guide — Endpoint PowerShell Deployment & Patch Automation Toolkit

Use `PSAUTO-Discovery.ps1` for admin-workstation/automation readiness, `PSAUTO-Validation.ps1` to test detection on a device, `PSAUTO-Remediation.ps1` for a safe fix, and `PSAUTO-Report.ps1` for fleet status.

A. Remediation never runs / no results

Symptom	Cause	Remediation
Device shows no detection result	Not synced, or not in the assigned group	Force a sync (Settings → Accounts → Access work or school → Info → Sync); confirm group membership
Remediation blade missing	Missing Remediations entitlement	Assign a qualifying license (see Requirements 2.1)
Script never fires	Schedule not set on the assignment	Edit the assignment and set `Endpoint Automation Schedule – Windows – Daily`

B. Detection always reports non-compliant

- The script is running 32-bit, so registry/file paths differ — set `Run script in 64-bit PowerShell`

to `Yes` on the remediation.

• The desired-state values at the top of `PSAUTO-Validation.ps1` do not match your standard — correct

them and re-upload the detection script.

• The check assumes the logged-on user's context but runs as SYSTEM — adjust the path or run as the

logged-on user only if the task truly requires it.

C. Remediation does not change anything

• It is running with the default `-WhatIf` (report-only). Re-run with `-WhatIf:$false` (or `-Apply`),

or set that default before uploading the remediation script.

• The remediation exits before acting because the device is already compliant — confirm with

`PSAUTO-Validation.ps1` that detection returns `1`.

D. Script blocked from running

Symptom	Cause	Remediation
Unsigned-script error	Execution policy is `AllSigned`	Sign the `.ps1` (Requirements 2.5), or set the remediation `Enforce script signature check = No` where policy allows
Access denied to a path/registry key	Running as the wrong context	Run as SYSTEM (default) for machine-wide paths; reserve logged-on-user for user-scope tasks

E. Fleet report errors

• `PSAUTO-Report.ps1` fails to connect — install the Graph SDK (Requirements 2.2) and consent to

`DeviceManagementManagedDevices.Read.All` and `DeviceManagementConfiguration.Read.All`.

• The report is empty — confirm the remediation is assigned and has run at least one cycle.

F. Deployment / cleanup runner issues

• Nothing installed/removed — it ran with the default `-WhatIf`; re-run with `-WhatIf:$false`.

• No log written — set `-LogPath` to a destination writable by the run context (Requirements 2.6).

• Cleanup left files — confirm the paths in the runner match the failed install's leftovers.

G. Diagnostics quick reference

```
Get-ExecutionPolicy -List                                    # execution policy per scope
[Environment]::Is64BitProcess                                # confirm 64-bit host
Get-Module -ListAvailable Microsoft.Graph.Authentication     # Graph SDK present?
```

```
Get-Content "$env:ProgramData\Microsoft\IntuneManagementExtension\Logs\AgentExecutor.log" -Tail 50
```

The IntuneManagementExtension `AgentExecutor.log` and `IntuneManagementExtension.log` on the device record remediation script output and exit codes.

H. When to escalate

If a device is enrolled, in the assigned group, synced, the remediation is assigned with a schedule, and the script still does not run, collect the `IntuneManagementExtension` logs above and open a Microsoft support case.

7. Best Practices — Endpoint PowerShell Deployment & Patch Automation Toolkit

• Always detect before you remediate. A detection script that exits `0`/`1` lets Intune fix only the devices that have drifted — never blindly run a change on every device.

• Pilot first, every time. Assign a new or changed remediation to `Endpoint Automation Pilot – Windows – Pilot` and pass the Testing Guide gate before adding the production group.

• Keep `-WhatIf` the default. The remediation and the deploy/cleanup runner should report planned actions before they change anything; flip to apply only after a clean pilot review.

• Make scripts idempotent. Running a remediation twice must be safe — check current state and exit early if the device is already healthy, so re-runs do no harm.

• Run in 64-bit as SYSTEM. Set `Run script in 64-bit PowerShell = Yes` so registry/file paths match;
default to SYSTEM unless the task genuinely needs the logged-on user.

• Keep remediations fast and non-interactive. Intune enforces a timeout; do the minimum work, never prompt, and never assume a UI is present.

• Write a clear status line and the right exit code. Every script should print what it found/did and return non-zero on failure so Intune and `PSAUTO-Report.ps1` catch problems.

• Name everything consistently. Use `<Purpose> – <Scope> – <Stage>` for packages, groups, and schedules, and reuse the names verbatim across Intune and your docs for auditability.

• Centralize the record. Point the deploy/cleanup runner's `-LogPath` at a share or workspace so you have a durable, trendable history beyond Intune's built-in results.

• Version and document your scripts. Record the desired-state values, the schedule, and the target groups so the automation is repeatable and auditable.

8. Deliverables Summary — Endpoint PowerShell Deployment & Patch Automation Toolkit

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The automation model, detect-then-remediate, scheduled runs, safe-by-default design, reporting
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Infrastructure)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: stand up the toolkit, deploy a detect/remediate pair, schedule, validate, expand
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations for the assign-once, repeat-everywhere model

PowerShell scripts

Script	What it does
`PSAUTO-Discovery.ps1`	Reports automation readiness: Graph SDK, execution policy, 64-bit host, remediations reachability
`PSAUTO-Validation.ps1`	Sample Intune remediation detection script; exits `0` compliant / `1` non-compliant
`PSAUTO-Report.ps1`	Fleet-wide remediation results summary across targeted devices (Microsoft Graph)
`PSAUTO-Remediation.ps1`	Sample remediation script that fixes drift; `-WhatIf` by default
`PSAUTO-Deploy-CleanupRunner.ps1`	Deployment/cleanup runner (install/remove/cleanup); `-WhatIf` by default

What the buyer receives

- This implementation guide as a branded Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable process following the model: Assess → Configure → Pilot → Validate → Expand.

Outcome

The repetitive endpoint work automated — detect-then-remediate jobs running on a schedule, results reported, and every device converging to one known-good state — proven on a pilot before you scale.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

PSAUTO-Deploy-CleanupRunner.ps1

Purpose: Runs a parameterized deployment or cleanup action on a device: install or uninstall an app (via its installer or product code) and remove leftover folders/files after a failed install. SAFE BY DEFAULT: runs in -WhatIf mode and only reports the actions it would take. Re-run with -WhatIf:$false to apply. Use -Action to choose Install, Uninstall, or Cleanup. HOW TO RUN Report-only (default): powershell -ExecutionPolicy Bypass -File .\PSAUTO-Deploy-CleanupRunner.ps1 -Action Install -InstallerPath "C:\Pkg\app.msi" -InstallArgs "/qn" Apply: powershell -ExecutionPolicy Bypass -File .\PSAUTO-Deploy-CleanupRunner.ps1 -Action Uninstall -ProductCode "{GUID}" -WhatIf:$false -LogPath "\\srv\share\psauto" powershell -ExecutionPolicy Bypass -File .\PSAUTO-Deploy-CleanupRunner.ps1 -Action Cleanup -CleanupPaths "C:\ProgramData\App\tmp","C:\Temp\App" -WhatIf:$false EXPECTED RESULTS -WhatIf (default): "WOULD: ..." lines, nothing changed, exit 0. Applied: "DID: ..." lines, the action performed, a log written to -LogPath if set, exit 0 on success / 1 on failure. .NOTES Endpoint automation toolkit — the repeatable way. Run as SYSTEM for machine-wide actions. #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param([ValidateSet('Install','Uninstall','Cleanup')][string]$Action = 'Cleanup', [string]$InstallerPath, [string]$InstallArgs = '/qn', [string]$ProductCode, [string[]]$CleanupPaths, [string]$LogPath)

How to run: Report-only (default): powershell -ExecutionPolicy Bypass -File .\PSAUTO-Deploy-CleanupRunner.ps1 -Action Install -InstallerPath "C:\Pkg\app.msi" -InstallArgs "/qn" Apply: powershell -ExecutionPolicy Bypass -File .\PSAUTO-Deploy-CleanupRunner.ps1 -Action Uninstall -ProductCode "{GUID}" -WhatIf:$false -LogPath "\\srv\share\psauto" powershell -ExecutionPolicy Bypass -File .\PSAUTO-Deploy-CleanupRunner.ps1 -Action Cleanup -CleanupPaths "C:\ProgramData\App\tmp","C:\Temp\App" -WhatIf:$false EXPECTED RESULTS -WhatIf (default): "WOULD: ..." lines, nothing changed, exit 0. Applied: "DID: ..." lines, the action performed, a log written to -LogPath if set, exit 0 on success / 1 on failure. .NOTES Endpoint automation toolkit — the repeatable way. Run as SYSTEM for machine-wide actions. #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param([ValidateSet('Install','Uninstall','Cleanup')][string]$Action = 'Cleanup', [string]$InstallerPath, [string]$InstallArgs = '/qn', [string]$ProductCode, [string[]]$CleanupPaths, [string]$LogPath)

PSAUTO-Discovery.ps1

Purpose: Reports whether this admin workstation / device is ready to build and run the toolkit: the Microsoft Graph PowerShell SDK, execution policy, 64-bit host, OS build, and whether the Intune Management Extension (which runs remediations) is present on the device. HOW TO RUN Elevated: powershell -ExecutionPolicy Bypass -File .\PSAUTO-Discovery.ps1 EXPECTED RESULTS A readiness summary: Graph SDK present, execution policy, 64-bit confirmed, OS build, and an Intune Management Extension status line. Warnings (yellow) flag anything to fix first. .NOTES Endpoint automation toolkit — the repeatable way. Read-only. #> [CmdletBinding()] param() Write-Host "`n=== Endpoint automation readiness ($($env:COMPUTERNAME)) ===" -ForegroundColor Cyan

How to run: Elevated: powershell -ExecutionPolicy Bypass -File .\PSAUTO-Discovery.ps1 EXPECTED RESULTS A readiness summary: Graph SDK present, execution policy, 64-bit confirmed, OS build, and an Intune Management Extension status line. Warnings (yellow) flag anything to fix first. .NOTES Endpoint

automation toolkit — the repeatable way. Read-only. #> [CmdletBinding()] param() Write-Host "`n=== Endpoint automation readiness ($($env:COMPUTERNAME)) ===" -ForegroundColor Cyan

PSAUTO-Remediation.ps1

Purpose: Brings a drifted device back to the desired known-good state. Use it as the remediation script in an Intune remediation; Intune runs it only when the detection script exited 1. SAFE BY DEFAULT: it runs in -WhatIf mode and only reports the changes it would make. Re-run with -WhatIf:$false (or set that default before uploading) to apply. The sample sets a registry value and starts a service; edit the DESIRED STATE block to match the detection script. HOW TO RUN In Intune: upload as the Remediation script in the same script package as PSAUTO-Validation.ps1. Run script in 64-bit PowerShell = Yes; run as SYSTEM (logged-on credentials = No). Locally (report-only): powershell -ExecutionPolicy Bypass -File .\PSAUTO-Remediation.ps1 Locally (apply): powershell -ExecutionPolicy Bypass -File .\PSAUTO-Remediation.ps1 -WhatIf:$false EXPECTED RESULTS -WhatIf (default): "WOULD: ..." lines and exit 0, nothing changed. Applied: "FIXED: ..." lines, the device set to the desired state, exit 0 on success / 1 on failure. .NOTES Endpoint automation toolkit — the repeatable way. Idempotent; safe to re-run. #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='Medium')] param()

How to run: In Intune: upload as the Remediation script in the same script package as PSAUTO-Validation.ps1. Run script in 64-bit PowerShell = Yes; run as SYSTEM (logged-on credentials = No). Locally (report-only): powershell -ExecutionPolicy Bypass -File .\PSAUTO-Remediation.ps1 Locally (apply): powershell -ExecutionPolicy Bypass -File .\PSAUTO-Remediation.ps1 -WhatIf:$false EXPECTED RESULTS -WhatIf (default): "WOULD: ..." lines and exit 0, nothing changed. Applied: "FIXED: ..." lines, the device set to the desired state, exit 0 on success / 1 on failure. .NOTES Endpoint automation toolkit — the repeatable way. Idempotent; safe to re-run. #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='Medium')] param()

PSAUTO-Report.ps1

Purpose: Summarizes how an Intune remediation (proactive remediation) is doing across the fleet: how many devices are without issues, remediated, or failed, so you can see convergence and chase laggards. Reads the remediation's per-device run states from Microsoft Graph. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\PSAUTO-Report.ps1 -ScriptName "Endpoint Drift Remediation - Windows - Production" -CsvPath C:\Temp\psauto.csv Requires the Microsoft Graph SDK and DeviceManagementConfiguration.Read.All + DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS A per-state count (Without issues / Remediated / Failed / Pending) for the named remediation and an optional per-device CSV (device, detection state, remediation state, last run time). .NOTES Endpoint automation toolkit — the repeatable way. Read-only against Graph. #> [CmdletBinding()] param([Parameter(Mandatory=$true)][string]$ScriptName, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\PSAUTO-Report.ps1 -ScriptName "Endpoint Drift Remediation - Windows - Production" -CsvPath C:\Temp\psauto.csv Requires the Microsoft Graph SDK and DeviceManagementConfiguration.Read.All + DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS A per-state count (Without issues / Remediated / Failed / Pending) for the named remediation and an optional per-device CSV (device, detection state, remediation state, last run time). .NOTES Endpoint automation toolkit — the repeatable way. Read-only against Graph. #> [CmdletBinding()] param([Parameter(Mandatory=$true)][string]$ScriptName, [string]$CsvPath)

PSAUTO-Validation.ps1

Purpose: Detects whether this device matches the desired known-good state. Use it as the detection script in an Intune remediation (proactive remediation). It exits 0 when the device is compliant (nothing to do) and exits 1 when it has drifted (Intune then runs the paired remediation, PSAUTO-Remediation.ps1). The sample checks a registry value and a service state; edit the DESIRED STATE block for your standard. HOW TO RUN In Intune: Devices > Manage devices > Remediations (Scripts and remediations) > Create script package > upload as the Detection script. Run in 64-bit PowerShell = Yes. Locally: powershell -ExecutionPolicy Bypass -File .\PSAUTO-Validation.ps1 EXPECTED RESULTS Exit 0 + "COMPLIANT" when state matches; exit 1 + "NON-COMPLIANT: " when it does not. .NOTES Endpoint automation toolkit — the repeatable way. Read-only (detection only; makes no changes). #> [CmdletBinding()] param()

How to run: In Intune: Devices > Manage devices > Remediations (Scripts and remediations) > Create script package > upload as the Detection script. Run in 64-bit PowerShell = Yes. Locally: powershell -ExecutionPolicy Bypass -File .\PSAUTO-Validation.ps1 EXPECTED RESULTS Exit 0 + "COMPLIANT" when state matches; exit 1 + "NON-COMPLIANT: <reason>" when it does not. .NOTES Endpoint automation toolkit — the repeatable way. Read-only (detection only; makes no changes). #> [CmdletBinding()] param()